EXP NO: 4A	SUPPORT VECTOR MACHINES (SVM)

AIM:

To implement a Support Vector Machine (SVM) classifier on the Breast Cancer dataset, compare different kernels, tune its hyperparameters, and evaluate it using accuracy, confusion matrix, classification report and ROC-AUC.

ALGORITHM:

1. Import the required libraries (numpy, pandas, matplotlib, seaborn, scikit-learn).
2. Load the Breast Cancer dataset (30 features, binary classes: malignant and benign).
3. Split the dataset into training (80%) and testing (20%) sets and scale the features using StandardScaler.
4. Train SVM models with the linear, polynomial, RBF and sigmoid kernels using SVC() and compare their accuracy and number of support vectors.
5. Build the SVM model with the RBF kernel, train it using fit() and predict the test data.
6. Evaluate the model using accuracy, ROC-AUC, confusion matrix and classification report.
7. Tune the hyperparameters C, gamma and kernel using GridSearchCV with 5-fold cross-validation.
8. Plot the kernel comparison and confusion matrix, and visualise the decision boundaries after reducing the data to 2 dimensions using PCA.

CODE:

```
# 1. Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix,
    classification_report, roc_auc_score
# 2. Load the dataset (Breast Cancer - binary classification)
```

```
data = load_breast_cancer()
X, y = data.data, data.target
print("Dataset shape:", X.shape)
print("Classes:", ", ".join(data.target_names))

# 3. Split and scale (SVM is sensitive to feature scale)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

# 4. Compare different kernels
print("\nKernel comparison:")
kernel_acc = {}
for k in ['linear', 'poly', 'rbf', 'sigmoid']:
    model = SVC(kernel=k, random_state=42).fit(X_train_s, y_train)
    kernel_acc[k] = accuracy_score(y_test, model.predict(X_test_s))
    print(f"  {k:8s} Accuracy = {kernel_acc[k]:.4f}   Support vectors =
        {model.n_support_.sum()}")

# 5. Build the SVM model with the RBF kernel and evaluate it
svm = SVC(kernel='rbf', C=1.0, gamma='scale', probability=True, random_state=42)
svm.fit(X_train_s, y_train)
y_pred = svm.predict(X_test_s)
print("\nSVM (RBF) Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("SVM (RBF) ROC-AUC :", round(roc_auc_score(y_test,
    svm.predict_proba(X_test_s)[:, 1]), 4))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n",
      classification_report(y_test, y_pred, target_names=data.target_names))

# 6. Hyperparameter tuning using GridSearchCV
param_grid = {'C': [0.1, 1, 10, 100], 'gamma': ['scale', 0.1, 0.01, 0.001],
              'kernel': ['linear', 'rbf']}
grid = GridSearchCV(SVC(), param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_train_s, y_train)
best_pred = grid.best_estimator_.predict(X_test_s)
print("Best Parameters:", grid.best_params_)
print("Best CV Accuracy:", round(grid.best_score_, 4))
print("Tuned SVM Test Accuracy:", round(accuracy_score(y_test, best_pred), 4))

# 7. Visualise: kernel accuracy and confusion matrix
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
ax[0].bar(kernel_acc.keys(), kernel_acc.values(), color=['#4c72b0', '#55a868',
    '#c44e52', '#8172b2'])
ax[0].set_ylim(0.8, 1.0); ax[0].set_ylabel("Test Accuracy");
    ax[0].set_title("SVM Kernel Comparison")
sns.heatmap(confusion_matrix(y_test, best_pred), annot=True, fmt='d',
    cmap='Blues', ax=ax[1],
            xticklabels=data.target_names, yticklabels=data.target_names)
ax[1].set_title("Tuned SVM Confusion Matrix"); ax[1].set_xlabel("Predicted");
    ax[1].set_ylabel("Actual")
plt.tight_layout()
plt.show()
```

```
# 8. Decision boundaries (data reduced to 2-D with PCA for plotting)
pca = PCA(n_components=2)
X2_train = pca.fit_transform(X_train_s)
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
for a, k in zip(ax, ['linear', 'rbf']):
    m = SVC(kernel=k).fit(X2_train, y_train)
    xx, yy = np.meshgrid(np.linspace(X2_train[:, 0].min() - 1, X2_train[:,
        0].max() + 1, 300),
                         np.linspace(X2_train[:, 1].min() - 1, X2_train[:,
                             1].max() + 1, 300))
    Z = m.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
    a.contourf(xx, yy, Z, alpha=0.3, cmap='coolwarm')
    a.scatter(X2_train[:, 0], X2_train[:, 1], c=y_train, cmap='coolwarm',
        edgecolor='k', s=20)
    a.scatter(m.support_vectors_[:, 0], m.support_vectors_[:, 1], s=80,
        facecolors='none', edgecolors='k')
    a.set_title(f"SVM Decision Boundary ({k} kernel)"); a.set_xlabel("PCA 1");
        a.set_ylabel("PCA 2")
plt.tight_layout()
plt.show()
```

OUTPUT:

```
Dataset shape: (569, 30)
Classes: malignant, benign

Kernel comparison:
  linear   Accuracy = 0.9737   Support vectors = 32
  poly     Accuracy = 0.9123   Support vectors = 145
  rbf      Accuracy = 0.9825   Support vectors = 97
  sigmoid  Accuracy = 0.9298   Support vectors = 66

SVM (RBF) Accuracy: 0.9825
SVM (RBF) ROC-AUC : 0.995

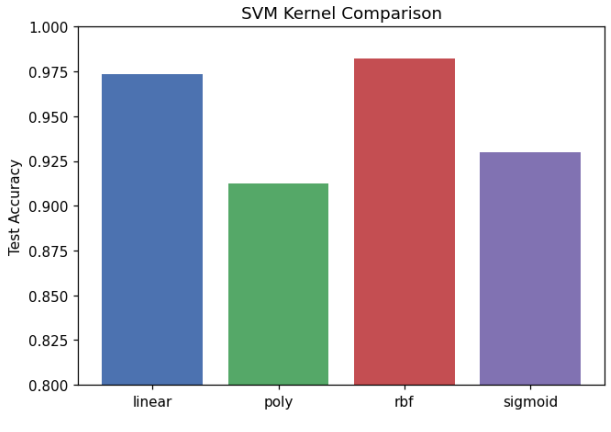
Confusion Matrix:
 [[41  1]
 [ 1 71]]

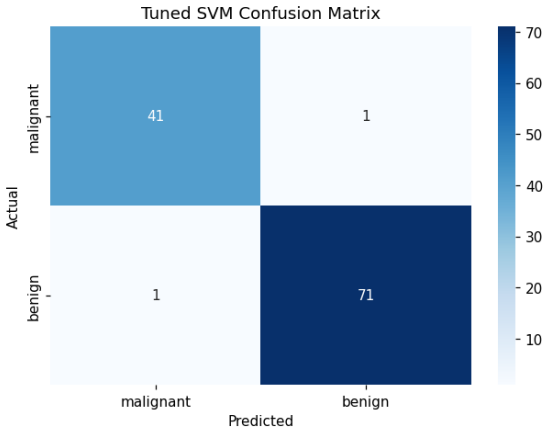
Classification Report:
              precision    recall  f1-score   support

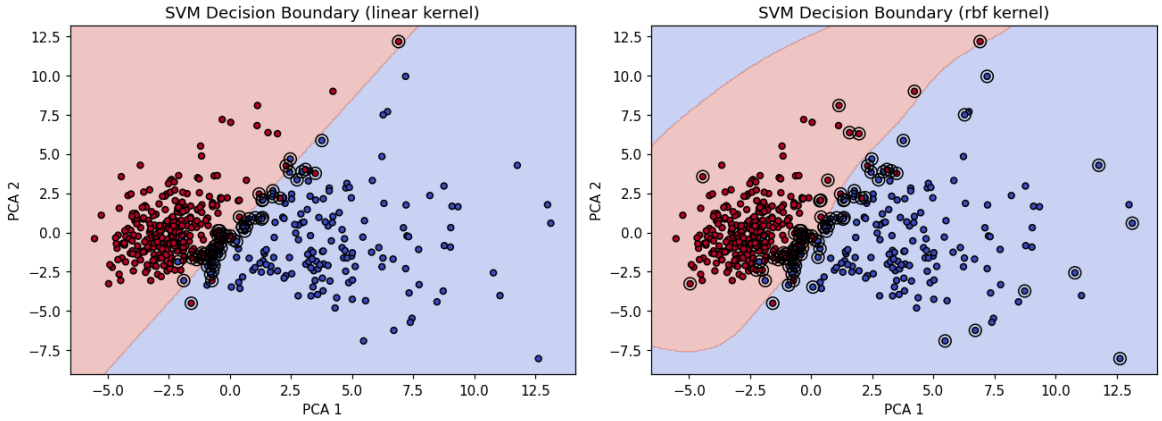
   malignant       0.98      0.98      0.98        42
      benign       0.99      0.99      0.99        72

    accuracy                           0.98       114
   macro avg       0.98      0.98      0.98       114
weighted avg       0.98      0.98      0.98       114

Best Parameters: {'C': 0.1, 'gamma': 'scale', 'kernel': 'linear'}
Best CV Accuracy: 0.9802
Tuned SVM Test Accuracy: 0.9825
```







RESULT:

The Support Vector Machine was successfully implemented on the Breast Cancer dataset. Among the kernels, RBF gave the highest test accuracy of 98.25% (ROC-AUC 0.995), followed by linear (97.37%), while polynomial (91.23%) and sigmoid (92.98%) performed worse. GridSearchCV selected the linear kernel with C=0.1 (cross-validation accuracy 98.02%) and the tuned model also achieved a test accuracy of 98.25%, with only two misclassified samples out of 114.